Isolamento de Processos e Limites de Recursos no Linux

SISTEMAS OPERACIONAIS – COMP0472 (2026.2 - AE)
Discentes: Augusto César Honorato dos Santos.
Docente: Prof Dr. Glauco de Figueiredo Carneiro

1. Objetivo

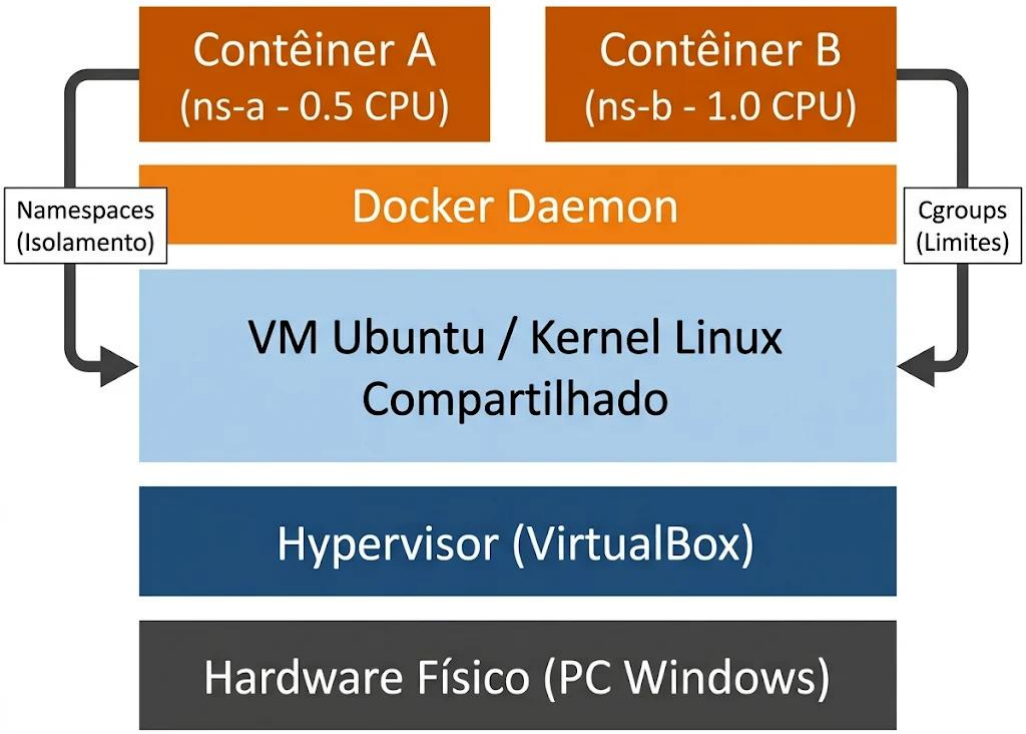
Investigar empiricamente o funcionamento dos namespaces (isolamento de visão) e cgroups v2 (limitação de recursos) no Kernel Linux, demonstrando as diferenças arquiteturais entre contêineres e Máquinas Virtuais (VMs).

---

2. Ambiente e Infraestrutura (Parte A)

Os experimentos foram conduzidos em uma Máquina Virtual (VirtualBox) rodando Ubuntu 24.04 LTS. A virtualização foi utilizada para isolar o ambiente de testes do hardware físico.

- **Host/VM:** 2 CPUs lógicas, 3.8 GB de RAM.
- **Kernel Linux:** Compartilhado entre o host virtual e todos os contêineres.
- **Docker Engine:** Versão 29.1.3 (utilizando Systemd como Cgroup Driver).
- **Cgroup:** v2 (unificado).
- **Imagem Base:** python:3.12-slim, fixada pelo digest sha256:5e2bdb1b0d542d5039cffb8181c65930093949194a65c365dfb1b0fe13a4fa5b



3. Evidências de Namespaces (Parte B)

Para provar o isolamento de visão, subimos dois contêineres independentes (ns-a e ns-b) executando sleep 3600. Os handles (identificadores únicos do Kernel) foram extraídos de /proc/1/ns/.

Recurso	Handle ns-a	Handle ns-b	Interpretação
PID	pid:[4026532275]	pid:[4026532345]	Isolamento da árvore de processos.
Mount	mnt:[4026532266]	mnt:[4026532342]	Isolamento da visão de sistema de arquivos.
Network	net:[4026532277]	net:[4026532347]	Isolamento de interfaces e rotas de rede.

Análise: Embora ambos os contêineres revelem um "PID 1 interno" (o processo sleep), o PID real alocado no host foi diferente (2888 para A, e 2942 para B). Isso demonstra que o namespace de PID cria uma nova árvore de numeração. O PID 1 do contêiner é uma **ilusão de visão**; ele não é o processo init do sistema operacional real. Os handles distintos provam fisicamente que as visões de mount e network são separadas.

---

4. Metodologia e Resultados (Cgroups e Throttling)

A carga de trabalho consistiu em um script Python determinístico (carga_cpu.py) executando 10.000.000 de iterações. Para analisar o comportamento do Kernel em tempo real, monitoramos o arquivo cpu.stat durante a execução da carga.

- **C1 (0.5 CPU):** O processo foi estrangulado (throttled) 69 vezes, acumulando um tempo de bloqueio de 4.989.906 microssegundos (~5 segundos pausado).
- **C2 (1.0 CPU):** O processo foi estrangulado 24 vezes, com tempo de bloqueio de 2.714.041 microssegundos (~2.7 segundos pausado).

Análise do Throttling: O mecanismo de *Completely Fair Scheduler (CFS)* do Kernel divide o tempo de CPU em fatias (períodos). Quando o contêiner C1 tentou ultrapassar os 50ms permitidos por período, o Kernel o forçou a dormir. O C2 também sofreu throttling porque o host (VM) possui apenas 2 núcleos e estava executando outras tarefas em paralelo, provando que o cgroup impõe um teto, mas não uma reserva exclusiva de hardware (afinidade).

---

5. Segurança e Mount Read-Only (Parte D)

Foi criado um diretório sintético (dados_demo) e montado com a flag :ro. O contêiner foi executado com o usuário 1000:1000, --cap-drop=ALL e --security-opt no-new-privileges.

- **Resultado:** A leitura do arquivo texto foi bem-sucedida. A tentativa de criar um novo arquivo retornou o erro Read-only file system.
- **Conclusão da segurança:** A negação da escrita é resultado do bloqueio do Kernel no *Virtual File System (VFS)*, aliado à remoção de privilégios. Em um cenário de servidor de inferência (IA), essa configuração é vital para evitar que um processo comprometido altere o corpus de dados ou os pesos do modelo.

---

6. Respostas às Questões de Análise (Seção 8 da Atividade)

1. **Onde está o kernel compartilhado? Há uma VM intermediária?**
   O kernel está no topo do hardware físico e da camada de virtualização. No nosso caso, há uma VM intermediária (VirtualBox). Todos os contêineres compartilham o *mesmo kernel* da VM. Isso significa que um bug crítico no kernel pode derrubar todos os contêineres simultaneamente, algo que não ocorre em VMs independentes (que têm kernels próprios).

2. **Quais diferenças de handles sustentam a separação? O que elas não demonstram?**
   A diferença nos handles de pid, mnt e net prova que a visão do sistema de arquivos, a rede e os processos estão isolados. No entanto, eles **não demonstram** limites de uso de CPU ou memória (isso é papel dos cgroups), nem garantem isolamento de privilégios (isso requer *user namespaces* e *capabilities*).

3. **Como diferenciar namespace, cgroup, identidade e política de acesso?**
   - **Namespace:** Muda o que o processo consegue *ver* (ilusão de que é a única máquina).
   - **Cgroup:** Limita o quanto o processo pode *usar* (teto de CPU, RAM e PIDs).
   - **Identidade (User):** Define *quem* é o processo e o que ele é autorizado a modificar no sistema de arquivos (ex: UID 1000).
   - **Política de Acesso (Capabilities/SELinux):** Define quais chamadas de sistema o processo pode fazer (ex: --cap-drop=ALL remove poderes de root).

4. **Como a quota foi confirmada? Houve throttling?**
   A quota foi confirmada via cpu.max. O throttling foi comprovado de forma contundente ao ler o cpu.stat *durante* a execução da carga real: no C1, o contador nr_throttled chegou a 69, mostrando que o Kernel interveio fisicamente 69 vezes

para pausar o processo e liberar a CPU. No C2, o número caiu para 24, mas não zerou, pois o ambiente virtualizado já possuía outras cargas consumindo os 2 núcleos disponíveis.

5. **Qual é a principal ameaça à validade do experimento?**
   A principal ameaça foi o ruído do host. Como a VM rodava em um computador de uso pessoal, outros processos do Windows ou da própria VM (como o contêiner open-webui que identificamos) podem ter consumido CPU em paralelo, alterando sutilmente os tempos do C2 e do C1.

6. **Por que limitar RAM/tarefas e montar corpus read-only importa em um servidor de inferência? Que risco permanece?**
   Limitar evita o "Efeito Dominó" (um modelo alucinado consumir toda a RAM e derrubar o servidor). O read-only impede a *Data Poisoning* (envenenamento de dados). O risco que permanece é a **vulnerabilidade de escape do contêiner** (Container Escape), onde uma falha no Kernel ou no Docker permite que o processo interno acesse o host real.

---

7. Limitações e Conclusão

O experimento comprovou que o contêiner **não é uma máquina virtual independente**. Ele é um processo comum no host, cuja visão foi alterada pelos namespaces e cujo consumo foi restrito pelos cgroups. A segurança do modelo depende da combinação de isolamento de visão, limites rígidos de recursos e remoção de privilégios, mantendo a necessidade crítica de atualização constante do kernel e do runtime (Docker) para evitar falhas de isolamento.